

Task 1

```
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt

import tensorflow as tf
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, UpSampling2D
from tensorflow.keras.models import Model
```

```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("paultimothymooney/blood-cells")

print("Path to dataset files:", path)
```

Using Colab cache for faster access to the 'blood-cells' dataset.
Path to dataset files: /kaggle/input/blood-cells

```
!cp -rf /root/.cache/kagglehub/datasets/paultimothymooney/blood-cells/versions/6 .
```

```
image_path = "/content/dataset-master/dataset-master/JPEGImages/BloodImage_00000.jpg"
```

```
image = cv2.imread(image_path)

# Convert BGR to RGB
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)

gray = cv2.resize(gray, (256, 256))
gray = gray.astype(np.float32) / 255.0
```

```
_, segmented = cv2.threshold(
    (gray * 255).astype(np.uint8),
    0,
    255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

segmented = segmented.astype(np.float32) / 255.0
```

```
input_image = np.expand_dims(gray, axis=-1)
input_image = np.expand_dims(input_image, axis=0)

print(input_image.shape)

(1, 256, 256, 1)
```

```
inputs = Input(shape=(256,256,1))

# Encoder
x = Conv2D(16, 3, activation='relu', padding='same')(inputs)
x = MaxPooling2D((2,2))(x)

x = Conv2D(32, 3, activation='relu', padding='same')(x)
x = MaxPooling2D((2,2))(x)

# Decoder
x = UpSampling2D((2,2))(x)
x = Conv2D(32, 3, activation='relu', padding='same')(x)

x = UpSampling2D((2,2))(x)
```

```
x = Conv2D(16, 3, activation='relu', padding='same')(x)

outputs = Conv2D(1, 1, activation='sigmoid')(x)

model = Model(inputs, outputs)

model.summary()
```

Model: "functional_7"

Layer (type)	Output Shape	Param #
input_layer_8 (InputLayer)	(None, 256, 256, 1)	0
conv2d_8 (Conv2D)	(None, 256, 256, 16)	160
max_pooling2d_2 (MaxPooling2D)	(None, 128, 128, 16)	0
conv2d_9 (Conv2D)	(None, 128, 128, 32)	4,640
max_pooling2d_3 (MaxPooling2D)	(None, 64, 64, 32)	0
up_sampling2d_2 (UpSampling2D)	(None, 128, 128, 32)	0
conv2d_10 (Conv2D)	(None, 128, 128, 32)	9,248
up_sampling2d_3 (UpSampling2D)	(None, 256, 256, 32)	0
conv2d_11 (Conv2D)	(None, 256, 256, 16)	4,624
conv2d_12 (Conv2D)	(None, 256, 256, 1)	17

Total params: 18,689 (73.00 KB)

```
prediction = model.predict(input_image)
plt.figure(figsize=(12,5))

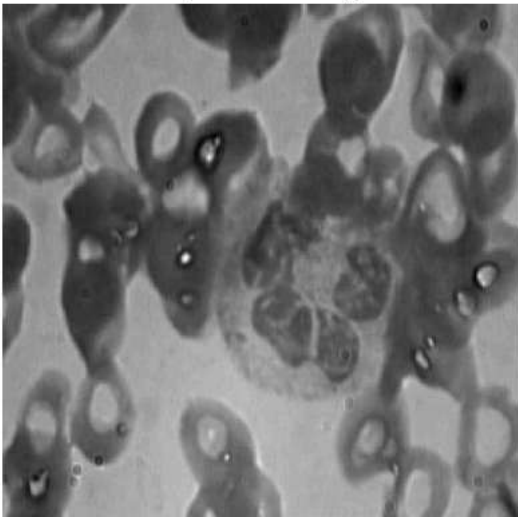
plt.subplot(1,2,1)
plt.imshow(gray, cmap='gray')
plt.title("Preprocessed Image")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(segmented, cmap='gray')
plt.title("Segmented Output")
plt.axis("off")

plt.show()
```

1/1 ————— 0s 254ms/step

Preprocessed Image



Segmented Output



Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.